GESTOR DE RUTAS TRANSMILENIO: Troncal Calle 26

Jeicob Stiven Murillo Ramos (1000334659), Andres Santiago Morales Robayo (1011080401)
Universidad Nacional de Colombia
Sede Bogotá
Bogotá D.C., Colombia
{jemurillor, andmoralesro}@unal.edu.co

***Resumen*—TransMilenio es un sistema compuesto por varias rutas, las cuales tienen sus paradas a lo largo de las estaciones de la ciudad, haciendo que no exista una única manera de ir de una estación A a una estación B. Para esta primera entrega nos concentramos en la troncal Calle 26 (estaciones y rutas que van por la Calle 26), la cual comprende desde Portal El Dorado hasta Universidades, proponiendo un programa en Java que indique qué rutas paran en cada estación, cuáles van directo entre dos estaciones y cuál es la mejor manera de viajar. Esta red se representa como un grafo, siendo las estaciones los vértices y las rutas las conexiones. El método usado para encontrar el mejor camino es el algoritmo de Dijkstra con un montículo de mínimos implementado por el equipo, y la búsqueda en anchura (BFS) para el camino con menos paradas. Se logró cargar las 18 rutas que van entre estaciones solamente por el carril exclusivo, respondiendo consultas por consola, y el programa pasa todas sus pruebas automáticas.**

***Index Terms*—estructuras de datos, grafos, algoritmo de Dijkstra, búsqueda en anchura, montículo binario, tablas hash, transporte público, TransMilenio**

I. Introducción

El principal sistema masivo de transporte de Bogotá es TransMilenio, el cual funciona a través de varias estaciones ubicadas a lo largo de la ciudad, en las cuales paran buses articulados tipo BRT (*Bus Rapid Transit*, autobús de tránsito rápido). Al ser un sistema tan grande, está dividido en troncales que se encuentran en diferentes zonas de la ciudad, conectadas entre sí.

Para esta entrega el caso de estudio es la troncal Calle 26, la cual consta de 14 estaciones (de Portal El Dorado hasta Universidades–City U) y 18 rutas que las recorren solamente por el carril exclusivo según la guía oficial de TransMilenio [5] (K86-M86, K54-H54, K53-G53, K43-S43, K23-B23, K16-B16, K10-L10, 3-3, 1-1).

I-A. Problema

Un usuario del sistema que, por ejemplo, está en Normandía y quiere ir hasta el Concejo de Bogotá necesita saber qué rutas paran en su estación, si alguna lo lleva directamente y, si no, en qué estación cambiar de bus. En las estaciones se muestran las rutas, pero no se responden estas preguntas.

I-B. Por qué es relevante

Al ser un sistema tan grande, es difícil aprenderse todas las rutas y, por ende, cuál sería la mejor en cada caso, lo que abre la posibilidad de trayectos innecesariamente largos y una mala experiencia del usuario. Aparte, en este problema se pueden aplicar los conceptos vistos a lo largo del curso: se resuelve con grafos, colas de prioridad, colas, pilas y tablas hash, y así mismo se puede hacer un análisis del costo de cada operación.

I-C. Dificultades

- Entre dos estaciones puede haber varias rutas, y cada ruta va en un solo sentido.
- Las rutas expresas se saltan estaciones, así que su tiempo entre dos paradas depende de cuántas paradas se omiten.
- El mejor viaje no es solo el más corto: cada cambio de bus implica esperar.
- El usuario escribe los nombres de varias maneras (con o sin tildes, o con otros nombres como “Maloka”).

I-D. Propuesta

Se propone hacer un programa en Java que represente el tramo como un grafo y responda a las necesidades del usuario desde la consola, indicando qué bus tomar, dónde bajarse y cuánto tarda el viaje aproximadamente.

II. OBJETIVOS Y ALCANCE

II-A. Objetivo general

Desarrollar un programa en Java que represente las estaciones y rutas (que solamente van por el carril exclusivo) de la troncal Calle 26 como un grafo y encuentre la mejor forma de viajar entre dos estaciones.

II-B. Objetivos específicos

1. Cargar estaciones y rutas desde archivos CSV y guardarlas en un grafo.
2. Consultar qué rutas paran en una estación y cuáles van directo entre dos estaciones.
3. Calcular la ruta más rápida, la de menos transbordos y la de menos paradas.
4. Justificar las estructuras de datos usadas y el costo de las operaciones.

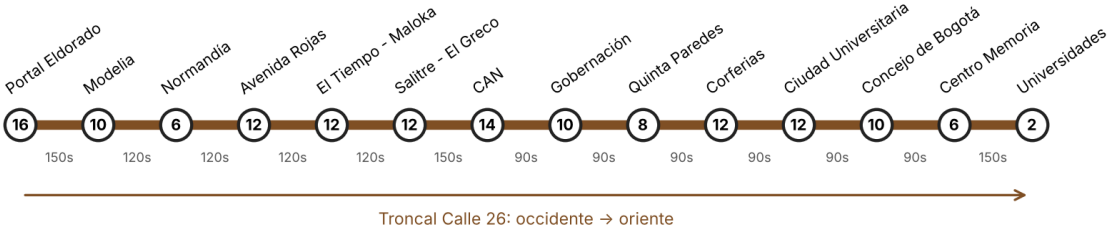


Figura 1. Tramo modelado. El número dentro de cada estación indica cuántas rutas paran en ella; entre cada tramo aparece su tiempo estimado.

II-C. Alcance

Incluye: las 14 estaciones del tramo Portal El Dorado – Universidades/City U, las 18 rutas (que van solamente por el carril exclusivo) que paran en ellas y una interfaz por consola. **No incluye:** horarios de las rutas, otras troncales, información en tiempo real ni interfaz gráfica.

II-D. Restricciones técnicas

- Lenguaje Java 17 o superior, sin bibliotecas externas.
- Interfaz por consola.
- Datos en archivos CSV con codificación UTF-8.
- Desarrollo durante el semestre, en tres entregas.

II-E. Supuestos

- Las rutas y sus paradas se tomaron de la guía oficial de TransMilenio [5].
- Los tiempos entre estaciones se estiman entre 1,5 y 2,5 minutos; se suman 30 segundos por parada y 4 minutos de espera en cada transbordo.
- Se considera que todas las rutas tienen la misma disponibilidad de horario.

II-F. Requerimientos funcionales

Cuadro I
Requerimientos funcionales

ID	Requerimiento	Estado
RF1	Cargar estaciones, tramos y rutas desde CSV.	Implementado
RF2	Mostrar las rutas que paran en una estación.	Implementado
RF3	Mostrar las paradas de una ruta.	Implementado
RF4	Mostrar las rutas directas entre dos estaciones.	Implementado
RF5	Calcular la ruta más rápida.	Implementado
RF6	Calcular la ruta con menos transbordos.	Implementado
RF7	Calcular la ruta con menos paradas.	Implementado
RF8	Buscar estaciones por nombre, sin importar tildes.	Implementado
RF9	Tener en cuenta los horarios de las rutas.	Pendiente
RF10	Autocompletar nombres de estaciones.	Pendiente

III. Trabajo relacionado

Encontrar la ruta más corta entre un nodo de origen y todos los demás es un problema clásico en movilidad. El algoritmo de Dijkstra [3] lo resuelve cuando los costos de sus conexiones son positivos, y la búsqueda en anchura (BFS) encuentra el camino con menos pasos cuando todos tienen el mismo costo [1].

Este método ya es usado por plataformas como Google Maps, Waze o Moovit, solo que estas ya tienen acceso a horarios detallados y técnicas avanzadas [4].

La diferencia con este proyecto es que es a menor escala y busca sobre todo la implementación de los conceptos vistos a lo largo del semestre.

IV. Diseño de solución

IV-A. Modelo

La red se divide en dos grafos:

- **Grafo de la vía:** une a cada estación con su vecina y guarda el tiempo de recorrido entre ellas (13 tramos, Fig. 1).
- **Grafo de rutas:** a cada ruta se le une cada parada con la siguiente. Respeta el sentido de dirección del bus (K16 va hacia el Portal y B16 en sentido contrario) y puede tener varias conexiones entre las mismas dos estaciones.

IV-B. Componentes

IV-C. La mejor ruta

Para encontrar la mejor ruta se usa el algoritmo de Dijkstra [3], el cual consiste en que desde el nodo de origen (estación inicial) se avanza siempre por la opción que lleve menos tiempo acumulado hasta llegar al destino. Los transbordos se tienen en cuenta cuando el algoritmo identifica que se viaja por una ruta y en la siguiente

Cuadro II
Componentes del programa

Clase	Responsabilidad
`CargadorDatos`	Lee los archivos CSV.
`RedTransmilenio`	Guarda los dos grafos y responde consultas sobre estaciones y rutas.
`CalculadorRutas`	Busca la mejor ruta (Dijkstra y BFS).
`MinHeap`	Cola de prioridad implementada por el equipo.
`Main`	Menú de consola.

conexión esta cambia; en ese caso se agregan 4 minutos de espera, y así se obtiene la ruta más rápida. Para la ruta con menos transbordos se usa el mismo algoritmo, pero con una penalización tan alta por cambiar de bus que siempre prefiere seguir en la misma ruta. La ruta con menos paradas se encuentra con la búsqueda en anchura (BFS), la cual hace una exploración por niveles y encuentra el camino con menos paradas.

IV-D. Ejemplo

Al ir de Normandía al Concejo de Bogotá, la respuesta del programa es:

```
RUTA MÁS RÁPIDA: Normandía -> Concejo de Bogotá
Tiempo total: 20 min 30 s | Transbordos: 0
  1. Tome 1 en Normandía y bájese en
     Concejo de Bogotá (9 paradas)
```

En cambio, la ruta con menos paradas solo hace 4 paradas, pero exige 2 transbordos y tarda 26 minutos.

V. Estructura de datos

Listas de adyacencia sobre `HashMap`. Guardan los dos grafos: cada estación tiene la lista de sus conexiones. Se eligieron en lugar de una matriz porque hay pocas conexiones por estación y porque entre dos estaciones puede haber varias rutas a la vez, algo que una matriz no representa bien. Recorrer las conexiones de una estación cuesta $O(\text{grado})$.

Montículo de mínimos (`MinHeap`). Lo usa Dijkstra para sacar siempre la opción con menor tiempo acumulado. Fue implementado por el equipo sobre un arreglo; insertar y extraer el mínimo cuestan $O(\log n)$ [2], mientras que con una lista normal extraer el mínimo costaría $O(n)$.

Cola (`ArrayDeque`). La usa BFS para visitar las estaciones en orden de llegada. Agregar y sacar cuestan $O(1)$.

Pila (`ArrayDeque`). Al terminar la búsqueda, el camino se conoce de atrás hacia adelante; la pila lo invierte en $O(k)$, con k el número de paradas.

Tablas hash auxiliares. Una relaciona cada estación con las rutas que paran en ella, para responder esa consulta en $O(1)$. Otra relaciona cada nombre (sin tildes) con su estación, para encontrar lo que escribe el usuario.

VI. Estado de la implementación

El programa tiene la capacidad de, por ejemplo, confirmar que por la estación CAN pasan 14 rutas (contando la ida y el regreso de la ruta 1) y que de Portal El Dorado a Corferias hay 5 rutas directas (M86, S43, B16, H54 y 1).

Cuadro III
Costo de las operaciones principales

Operación	Estructura	Costo
Rutas que paran en una estación	Tabla hash	$O(1)$
Buscar estación por nombre	Tabla hash	$O(1)$ prom.
Insertar / extraer mínimo	Montículo	$O(\log n)$
Ruta más rápida	Dijkstra + montículo	$O(E \log E)$
Ruta con menos paradas	BFS + cola	$O(V + E)$
Reconstruir el camino	Pila	$O(k)$

V: estaciones; E: conexiones revisadas en la búsqueda; k: paradas.

Cuadro IV
Avance de la primera entrega

Funcionalidad	Estado
Carga de estaciones y rutas desde CSV	Implementado
Rutas por estación y rutas directas	Implementado
Ruta más rápida y con menos transbordos	Implementado
Ruta con menos paradas	Implementado
Menú de consola	Implementado
Búsqueda de estaciones por fragmento	Parcial
Horarios de las rutas	Pendiente
Autocompletado de nombres	Pendiente

VI-A. Pruebas

El repositorio incluye un programa de pruebas automáticas que revisa el montículo, la carga de datos, las consultas de rutas y los tres tipos de búsqueda. Todas las pruebas pasan. Una de ellas compara los resultados de Dijkstra con otro método más lento (Bellman-Ford) en todas las combinaciones de estaciones, y ambos coinciden.

VI-B. Primeros Resultados

En el tramo, todos los viajes posibles entre dos estaciones tienen al menos una ruta directa, principalmente gracias a la ruta 1, que para en las 14 estaciones en ambos sentidos. También se comprobó que tener en cuenta la espera es importante: sin ella, de Portal El Dorado a Universidades el programa proponía cambiar de bus cinco veces (47,5 minutos); con ella propone tomar directamente la ruta 1 (31 minutos).

VII. Conclusiones y próximos pasos

Para esta primera entrega se hizo el planteamiento del problema, el cual se soluciona por medio de grafos, y se hizo una primera versión funcional con las 18 rutas de la troncal Calle 26 (del Portal El Dorado a Universidades/City U), aplicando diferentes estructuras de datos, lo cual nos permitió responder las consultas de forma rápida y explicar cada resultado.

Para las siguientes entregas se propone:

1. Tener en cuenta los horarios, porque algunas rutas solo operan en ciertas franjas (por ejemplo, K53 y G53).
2. Agregar autocompletado de estaciones.
3. Ampliar la red a otras troncales.

Agradecimientos

Los autores agradecen a la Universidad Nacional de Colombia, Sede Bogotá, y a las personas o instituciones que apoyaron el desarrollo de este proyecto.

Referencias

[1] T. H. Cormen, C. E. Leiserson, R. L. Rivest y C. Stein, *Introduction to Algorithms*, 4th ed. Cambridge, MA, USA: MIT Press, 2022.

[2] M. T. Goodrich, R. Tamassia y M. H. Goldwasser, *Data Structures and Algorithms in Java*. Hoboken, NJ, USA: Wiley, 2014.

[3] E. W. Dijkstra, "A note on two problems in connexion with graphs," *Numerische Mathematik*, vol. 1, pp. 269–271, 1959.

[4] H. Bast, D. Delling, A. Goldberg, M. Müller-Hannemann, T. Pajor, P. Sanders, D. Wagner y R. F. Werneck, "Route planning in transportation networks," en *Algorithm Engineering*, LNCS vol. 9220. Cham, Suiza: Springer, 2016, pp. 19–80.

[5] TransMilenio S.A., "Guía general de viaje de TransMilenio, corte a agosto de 2026." [En línea]. Disponible en: https://www.transmilenio.gov.co/